

1. Different Types of Repositories

Overview

Repositories are design patterns that provide an abstraction layer between the business logic and data storage. They encapsulate the logic required to access data sources and present a collection-like interface to the domain model.

Types of Repositories

1. CRUD Repository (Basic Repository)

The most fundamental type that provides standard Create, Read, Update, Delete operations.

Key Operations:

- Save (Create/Update)
- Find by ID (Read)
- Find all
- Delete by ID
- Count records

Example:

java

```
// Custom CRUD Repository
public interface UserRepository extends CrudRepository<User, Long> {
    // Inherits all CRUD operations
}

// Usage
User user = new User("John", "john@email.com");
userRepository.save(user);           // Create
Optional<User> found = userRepository.findById(1L); // Read
userRepository.deleteById(1L);      // Delete
long count = userRepository.count(); // Count
```

2. Paging and Sorting Repository

Provides pagination and sorting capabilities on top of CRUD operations.

Key Features:

- Pagination with page numbers and page sizes

- Sorting by multiple fields
- Returns Page objects containing data and metadata

Example:

java

```
// Paging and Sorting Repository
public interface ProductRepository extends
PagingAndSortingRepository<Product, Long> {
}

// Usage
Pageable pageable = PageRequest.of(0, 10,
Sort.by("price").descending());
Page<Product> page = productRepository.findAll(pageable);

List<Product> products = page.getContent();
int totalPages = page.getTotalPages();
long totalElements = page.getTotalElements();
int currentPage = page.getNumber();
```

3. JPA Repository

Extends PagingAndSortingRepository and adds JPA-specific methods.

Additional Features:

- Flush operations
- Batch operations
- Entity management methods

Example:

java

```
// JPA Repository
public interface EmployeeRepository extends JpaRepository<Employee,
Long> {
}

// Usage
Employee emp = new Employee("Alice", "IT");
employeeRepository.saveAndFlush(emp); // Save and immediately flush
```

```
employeeRepository.deleteInBatch(employeeList); // Batch delete
employeeRepository.flush(); // Force flush to database
```

4. Custom Repository (Implementation-based)

Custom repositories with custom business logic implementations.

Example:

java

```
// Interface
public interface CustomUserRepository {
    List<User> findUsersByComplexCriteria(String name, String email, int
age);
}

// Implementation
@Repository
public class CustomUserRepositoryImpl implements CustomUserRepository {
    @PersistenceContext
    private EntityManager entityManager;

    @Override
    public List<User> findUsersByComplexCriteria(String name, String
email, int age) {
        String jpql = "SELECT u FROM User u WHERE u.name LIKE :name AND
u.email LIKE :email AND u.age > :age";
        return entityManager.createQuery(jpql, User.class)
            .setParameter("name", "%" + name + "%")
            .setParameter("email", "%" + email + "%")
            .setParameter("age", age)
            .getResultList();
    }
}
```

5. Reactive Repository (Spring Data MongoDB Reactive)

For reactive/non-blocking applications using Project Reactor.

Example:

java

```
@Repository
public interface ReactiveUserRepository extends
```

```

ReactiveMongoRepository<User, String> {
    Flux<User> findByName(String name);
    Mono<User> findByEmail(String email);
}

// Usage
reactiveUserRepository.findByName("John")
    .subscribe(user -> System.out.println("Found user: " + user));

```

6. QueryDSL/Spring Data JPA DSL Repository

Enables type-safe query building using a fluent API.

Example:

java

```

public interface QueryDSLUserRepository extends
QuerydslPredicateExecutor<User> {
}

// Usage
QUser user = QUser.user;
Predicate predicate = user.name.eq("John").and(user.age.gt(18));
Iterable<User> results = userRepository.findAll(predicate);

```

2. Query Methods in Spring Data JPA

Overview

Query methods are methods defined in repository interfaces that are automatically implemented by Spring Data JPA based on their method names. They eliminate boilerplate code by deriving queries from method signatures.

Types of Query Methods

1. Derived Query Methods (Method Naming Conventions)

Spring Data JPA parses method names and creates queries automatically.

Keywords and Usage:

Finding Methods

java

```

public interface UserRepository extends JpaRepository<User, Long> {

    // Find by property
    List<User> findByName(String name);

    // Find by property and another property
    List<User> findByNameAndEmail(String name, String email);

    // Find with LIKE operator
    List<User> findByNameLike(String pattern);

    // Find with IN clause
    List<User> findByNameIn(List<String> names);

    // Find with ORDER BY
    List<User> findByNameOrderByAgeAsc(String name);

    // Find with limiting results
    User findFirstByName(String name);
    List<User> findTop5ByName(String name);

    // Find with distinct
    List<User> findDistinctByName(String name);
}

```

Count Methods

java

```

public interface UserRepository extends JpaRepository<User, Long> {
    long countByName(String name);
    long countByNameAndAge(String name, int age);
}

```

Delete Methods

java

```

public interface UserRepository extends JpaRepository<User, Long> {
    void deleteByName(String name);
}

```

```
        void deleteByNameAndEmail(String name, String email);
    }
```

Exists Methods

java

```
public interface UserRepository extends JpaRepository<User, Long> {
    boolean existsByName(String name);
    boolean existsByNameAndEmail(String name, String email);
}
```

2. @Query Annotation (JPQL)

Use custom JPQL queries for complex logic.

Example:

java

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Simple JPQL query
    @Query("SELECT u FROM User u WHERE u.age > :age")
    List<User> findUsersOlderThan(@Param("age") int age);

    // Join query
    @Query("SELECT u FROM User u JOIN u.orders o WHERE o.total > :amount")
    List<User> findUsersWithOrdersOver(@Param("amount") double amount);

    // Update query
    @Modifying
    @Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
    int updateUserStatus(@Param("id") Long id, @Param("status") String status);

    // Complex projection
    @Query("SELECT new com.example.dto.UserDTO(u.id, u.name, u.email) FROM User u WHERE u.id = :id")
    UserDTO findUserDTOById(@Param("id") Long id);
}
```

3. Native Queries

Use raw SQL queries for database-specific operations.

Example:

java

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Native SQL query
    @Query(value = "SELECT * FROM users u WHERE u.name LIKE %:name%",
nativeQuery = true)
    List<User> searchByNameNative(@Param("name") String name);

    // Native query with joins
    @Query(value = "SELECT u.* FROM users u JOIN orders o ON u.id =
o.user_id WHERE o.total > :amount",
nativeQuery = true)
    List<User> findUsersWithHighOrderAmountNative(@Param("amount")
double amount);
}
```

4. Named Queries

Predefined queries defined in entity classes.

Example:

java

```
// In Entity class
@Entity
@NamedQueries({
    @NamedQuery(
        name = "User.findByAgeRange",
        query = "SELECT u FROM User u WHERE u.age BETWEEN :minAge AND
:maxAge"
    ),
    @NamedQuery(
        name = "User.findByStatus",
        query = "SELECT u FROM User u WHERE u.status = :status"
    )
})
public class User {
```

```

        // Entity fields
    }

    // In Repository
    public interface UserRepository extends JpaRepository<User, Long> {
        List<User> findByAgeRange(@Param("minAge") int minAge,
            @Param("maxAge") int maxAge);
        List<User> findByStatus(@Param("status") String status);
    }

```

5. Sorting and Pagination in Query Methods

Combine query methods with sorting and pagination.

Example:

java

```

public interface UserRepository extends JpaRepository<User, Long> {

    // With sorting
    List<User> findByNameContaining(String name, Sort sort);

    // With pagination
    Page<User> findByAgeGreaterThan(int age, Pageable pageable);

    // Combined with complex query
    @Query("SELECT u FROM User u WHERE u.age > :age")
    Page<User> findOlderUsers(@Param("age") int age, Pageable pageable);
}

// Usage
Pageable pageable = PageRequest.of(0, 10, Sort.by("name").ascending());
Page<User> page = userRepository.findOlderUsers(18, pageable);

```

6. Streaming Results

Process large datasets using Java Streams.

Example:

java

```

public interface UserRepository extends JpaRepository<User, Long> {

    @Query("SELECT u FROM User u WHERE u.age > :age")

```

```

        Stream<User> streamUsersOlderThan(@Param("age") int age);

        // With paging
        Stream<User> findAllByName(String name);
    }

    // Usage with try-with-resources
    try (Stream<User> stream = userRepository.streamUsersOlderThan(18)) {
        stream.filter(u -> u.getEmail().contains("@gmail.com"))
            .forEach(System.out::println);
    }

```

7. Asynchronous Query Methods

Non-blocking repository methods.

Example:

java

```

@Async
public interface UserRepository extends JpaRepository<User, Long> {

    @Async
    @Query("SELECT u FROM User u WHERE u.email LIKE :domain")
    CompletableFuture<List<User>>
    findByEmailDomainAsync(@Param("domain") String domain);

    @Async
    ListenableFuture<List<User>> findByAgeGreaterThan(int age);
}

// Usage
CompletableFuture<List<User>> future = userRepository
    .findByEmailDomainAsync("@example.com");
future.thenAccept(users -> System.out.println("Found users: " +
    users.size()));

```

Summary Table

Method Type	When to Use	Example

Derived Query	Simple queries with standard conditions	<code>findByN ameAndAg e()</code>
@Query (JPQL)	Complex queries with joins, projections	<code>@Query("SELECT u FROM User u JOIN u.orders o WHERE o.total > :amount")</code>
Native Query	Database-specific features, performance	<code>@Query(value = "SELECT * FROM users WHERE active = 1", nativeQu ery = true)</code>
Named Query	Reusable queries, clean	<code>@NamedQ uery(nam e = "User.by</code>

	entity separation	Age", query = "...")
Paged/Sorted	Large datasets, UI pagination	Page<User> findAll(Pageable pageable)
Streaming	Large datasets, efficient processing	Stream<User> findAll()
Async	Non-blocking operations	CompletableFuture<List<User>> findAll()

These different repository types and query methods provide powerful flexibility for data access while maintaining clean, maintainable code.